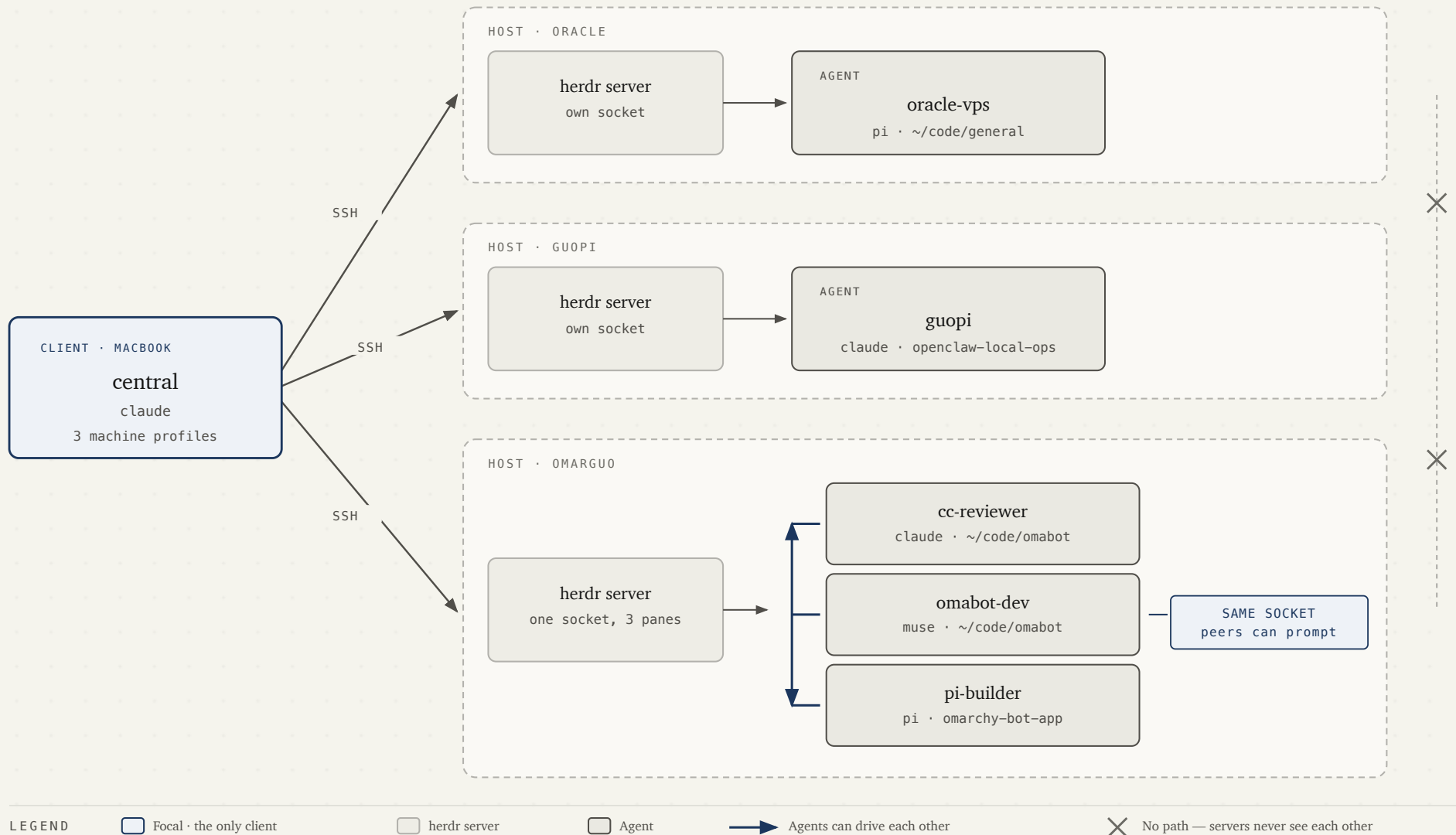


Five agents, four machines, one client

Every host runs its own herdr server and its agents register natively there. The MacBook is the only thing that can see all of them — and that is a deliberate property, not a gap.

Figure 1

How the fleet is wired



One focal node: `central` is the only client, and it holds all three machine profiles. Each host owns its own server and socket. The ink-blue bracket on

omarguo is the second thing worth looking at — three agents sharing one socket is the only place peer-to-peer works.

Section 1

Who is in the fleet

Verified live. Liveness is read from the terminal title, not the status — a dead agent still reports `idle`.

AGENT	HOST	KIND	WORKING DIRECTORY	SESSION
central	MacBook	claude	~/tools/commandcenter	client
oracle-vps	oracle	pi	~/code/general	reported
guopi	guopi	claude	~/codex/openclaw-local-ops	reported
cc-reviewer	omarguo	claude	~/code/omabot	reported
omabot-dev	omarguo	muse	~/code/omabot	n/a
pi-builder	omarguo	pi	~/code/omarchy-bot-app	reported

Why `omabot-dev` shows `n/a`: herdr can start and detect `muse`, but ships no integration for it, so it never reports a session id. Expected, not a fault. Re-check `herdr integration` after a herdr upgrade in case one lands.

Can agents talk to each other?

Yes on the same host. No across hosts. The difference is one environment variable.

■ WORKS · SAME HOST

Peers on one socket

Every pane herdr owns gets the same socket injected, so agents on one host are peers and can drive each other directly:

```
HERDR_ENV=1
HERDR_PANE_ID=w1:p3
HERDR_SOCKET_PATH=/home/mguo/.config/herdr/herdr.sock
```

That is why `cc-reviewer` can supervise `pi-builder` — it genuinely calls `herdr agent prompt pi-builder "..."`, and why it once hit `agent_not_found` when that agent restarted and lost its name.

□ DOES NOT WORK · ACROSS HOSTS

Three islands

A server knows only its own panes and never looks for peers. `oracle-vps` has no path to `guopi`, and saving a machine profile would not create one — the CLI inside a pane always talks to that pane's own server.

herdr machine add feeds the TUI sidebar, not agent-to-agent reach. Cross-host collaboration goes through `central` as a relay, or through ssh credentials granted deliberately, one edge at a time.

This is the same property that makes the trust direction safe: each client decides who *it* dials, and nothing is pushed onto a host from outside. A mesh would mean every host holding credentials for every other host.

Section 3

Adding a host, adding an agent

A new host

One command. It prepares the remote install, **starts its server**, and saves a profile holding only a label, ssh target, session name and enabled flag — auth stays with OpenSSH.

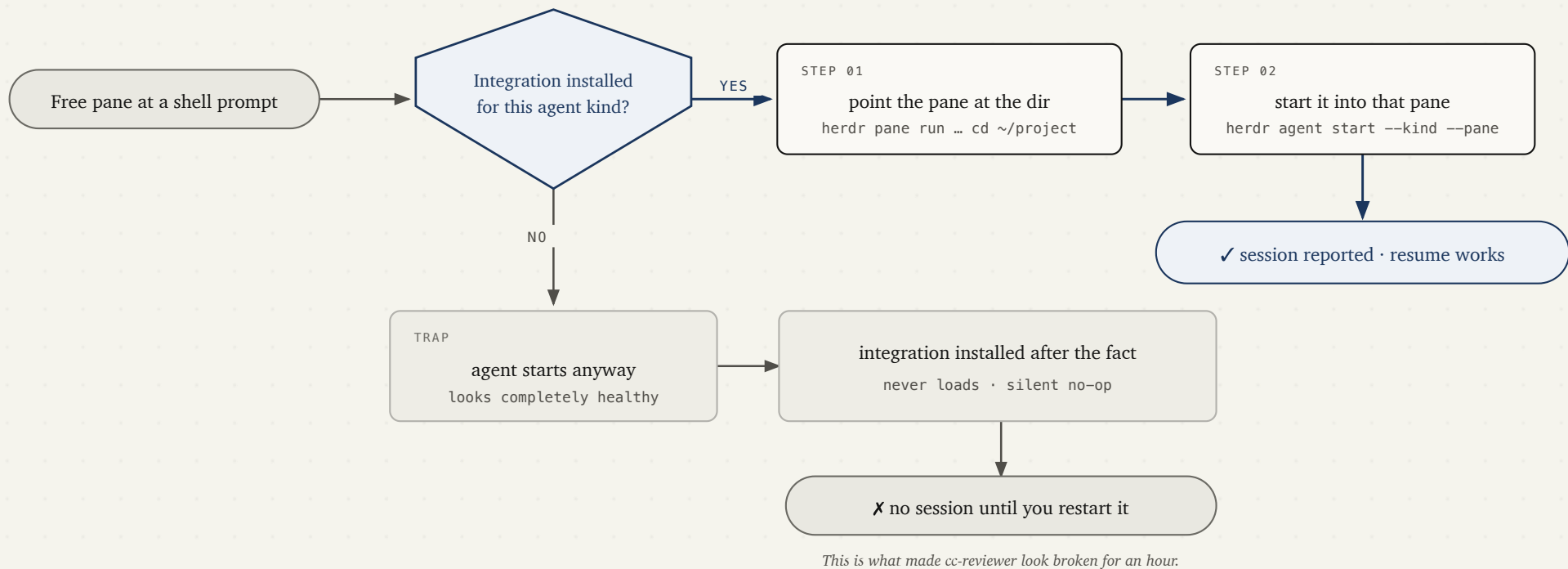
ON THE MACBOOK

```
$ herdr machine add <ssh-target> --label <label>
Saved SSH machine 02a7f066... Remote server is ready.
Open Herdr clients connect automatically.
```

The first argument is an *ssh target*, not a hostname — `oracle` and `guopi` work bare because they are `Host` entries in `~/.ssh/config`. Pass no `--remote-session` and it attaches to the remote's `default` session, which matters on a host that already has live agents.

A new agent on that host

1. **Install the integration first.** An integration only loads when the agent starts, so installing it under a running agent does nothing.
2. **Point a free pane at the working directory**, then start the agent into it. `agent start` has no `--cwd` ; it inherits the pane's.
3. **Name it for its role or host**, never its model — the model behind a name changes.



◇ Focal · the decision that traps people □ Step you run □ Failure path → Correct order

Order is the whole diagram. Both paths start an agent that looks healthy and reports `idle`. Only the top one reports a session id, because a herdr integration is read at agent startup and never again. Nothing warns you when you get it backwards.

ON THE TARGET HOST

```
$ herdr integration install claude          # before starting, not after
$ herdr pane run w1:p1 "cd ~/project"
$ herdr agent start my-agent --kind claude --pane w1:p1
```

Names are volatile. They clear whenever a pane's occupant is replaced — a restart, a crash, an upgrade. After any of those, re-run `herdr agent rename`. An unnamed agent is unaddressable: `herdr agent prompt` fails with `agent_not_found`.

Section 4

Launching them all

`commandcenter-up` registers missing machines, starts missing agents in their intended directory, and leaves anything already correct alone. It is idempotent — running it twice produces identical output and changes nothing.

The whole fleet is one table at the top of the script. Adding a host means adding a row.

 `commandcenter-up`

```
1 # label | ssh target | agent kind | cwd under $HOME | agent name
2 FLEET='oracle|oracle|pilcode/general|oracle-vps
3 guopi|guopi|claudel|codex/openclaw-local-ops|guopi'
4
5 # omarguo runs three long-lived agents, started and resumed on that
6 # host. We verify all three are present and never restart them.
7 OMARGUO_TARGET='mguo@omarguo.local'
8 OMARGUO_AGENTS='cc-reviewer pi-builder omabot-dev'
```


VERIFY WITHOUT CHANGING ANYTHING

```
$ ./commandcenter-up --check
machines ...
  ok    oracle
  ok    guopi
  ok    omarguo
agents ...
  ok    oracle-vps - pi idle (~code/general)
  ok    guopi - claude idle (~codex/openclaw-local-ops)
omarguo (verify only) ...
  ok    cc-reviewer - claude idle
  ok    pi-builder - pi idle
  ok    omabot-dev - muse idle
```

omarguo is **verify-only**: its three agents hold long-lived context, so the script confirms they are present and never restarts them. That is also the check that caught `pi-builder` silently losing its name after an unattended restart.

Design: **kami** — warm parchment, single ink-blue accent, serif hierarchy. The architecture figure is a hand-assembled inline SVG on kami's diagram tokens, one focal node, nine nodes inside the budget. Fleet data is live output from this session, not illustrative.